

Viva: Project 13, Anime Browser

API: Jikan (<https://jikan.moe>) - Key: No key needed

Brief: Top and seasonal anime with search and a detail screen.

30 questions, roughly easy to hard. Sections A and B are common ground for every project; C and D are specific to this one. Ask 8 to 12 in a typical 10 minute viva.

A. Kotlin and Android basics

1. What is the difference between val and var, and which did you use for your Retrofit instance?
2. What is a data class, and what does Kotlin generate for you?
3. How did you model an anime, and which of its many fields did you keep?
4. What does ? mean on a property, and which anime fields are genuinely optional?
5. How do you pass the selected anime to the detail screen?
6. How does a RecyclerView adapter know what to draw for each row?
7. Where do you declare the INTERNET permission, and what happens if you forget it?

B. Retrofit, coroutines and networking

8. What problem does Retrofit solve for you?
9. Explain the role of the converter factory in your Retrofit builder.
10. Which annotations carry the search query and the page number?
11. Why is your API call function marked suspend?
12. Which dispatcher runs the network call, and which updates the UI?
13. What happens if you call the API on the main thread?
14. What is a coroutine scope, and which one did you launch from?
15. How does your code behave if a page request fails midway through scrolling?

C. The Jikan API and this app

16. Which endpoints did you use for top anime and for search, and how do they differ?
17. Jikan wraps everything in a data field. How is that reflected in your Kotlin models?
18. Images are nested as images.jpg.image_url. How did you model that nesting, and was it worth it?
19. Jikan rate limits to roughly 3 requests per second and 60 per minute. How does your app respect that?
20. What happens when you exceed the limit, and what status code comes back?
21. pagination.has_next_page comes with each response. How did you use it?
22. Jikan is an unofficial wrapper around MyAnimeList and caches heavily. What does that mean for data freshness?
23. score and episodes are often null for unaired shows. How does your UI handle that?
24. The synopsis can be very long. How did you present it without breaking the layout?

D. Edge cases, design and extensions

25. What does your app show while the list is loading?

26. A user has no connection and opens the app. Describe exactly what they see.
27. Should you search on every keystroke given the rate limit? What is debouncing?
28. How would you add infinite scroll, and what stops duplicate page requests?
29. How would you add seasonal browsing, and which endpoint gives it?
30. How would you cache the top anime list so the app opens instantly?

Humble Coders project viva preparation. These are the kinds of questions you should be able to answer about the app you built. Knowing the list is not the same as knowing your own code.